Course Project Specification Document: Modern Web Technologies

1. Project Overview

This course project requires students to design, implement, integrate, secure, and deploy a **full-stack JavaScript web application** using modern web technologies. The project simulates a professional full-stack development engagement and emphasizes industry-aligned practices.

Students will apply advanced concepts in server-side JavaScript development, RESTful API design, NoSQL data modeling, authentication and authorization, frontend–backend integration, and deployment.

The project constitutes a **major summative assessment** and contributes **30 percent of the final course grade**.

2. Project Purpose

The purpose of this project is to enable students to:

- Apply modern web technologies to real-world software development scenarios
- Demonstrate full-stack development skills using Node.js, Express.js, MongoDB, and React
- Design secure, scalable, and maintainable backend systems
- Integrate frontend applications with backend services using REST APIs
- Deploy and validate a production-ready web application
- Demonstrate professional software development practices

3. Project Scope

Each project group must develop a **complete full-stack web application** that includes:

- A backend implemented using Node.js and Express.js
- A MongoDB database integrated using Mongoose
- A frontend application implemented using React
- Secure authentication and authorization mechanisms

- Deployment to a hosting or cloud-based environment

The application must support **multiple user roles**, enforce **business rules**, and manage **persistent data**.

4. Business Case Requirements

Each group must define an **original business scenario** that justifies the development of a secure, database-driven web application.

4.1 Business Case Complexity

The business case must:

- Address a real-world operational problem
- Require persistent data storage with meaningful relationships
- Support multiple user roles with distinct access privileges
- Include business workflows involving data lifecycle management
- Require backend validation and enforcement of business rules

Overly simplistic or tutorial-style applications will not be accepted.

4.2 Required Business Workflows

The system must support, at minimum:

1. **User Authentication Workflow**
    - Account creation and secure login
    - Token or session management
    - Access to protected resources
2. **Role-Based Workflow**
    - Privileged operations restricted by user role
    - Backend-enforced authorization rules
    - Explicit rejection of unauthorized actions
3. **Data Management Workflow**

- Creation, modification, and deletion of data
- Validation rules enforced at the backend
- Structured error handling and feedback

5. Technical Requirements

5.1 Mandatory Technology Stack

The following technologies must be used:

- Node.js
- Express.js
- MongoDB
- Mongoose
- React
- JSON Web Tokens
- GitHub
- Postman

Use of alternative frameworks or technologies requires prior instructor approval.

5.2 Backend Requirements

The backend application must:

- Follow a structured architecture separating routes, controllers or services, models, and middleware
- Provide RESTful API endpoints using appropriate HTTP methods
- Implement centralized error handling
- Use environment variables for configuration

Minimum requirements include:

- At least **10 RESTful API endpoints**

- At least **3 protected endpoints**
- Middleware for authentication, validation, and error handling

5.3 Database Requirements

The database design must include:

- A minimum of **6 MongoDB collections**
- Clearly defined schemas with validation rules
- Required fields and appropriate default values
- Meaningful relationships between collections
- Indexing where appropriate for performance

5.4 Security Requirements

The application must implement:

- Secure password hashing
- Token-based authentication using JSON Web Tokens
- Role-based authorization enforced at the backend
- Protection of sensitive routes and resources
- Secure management of environment variables

Frontend-only security implementations will not be accepted.

5.5 Frontend Requirements

The frontend application must:

- Be implemented using React
- Communicate exclusively with backend APIs
- Reflect role-based access control
- Provide validation feedback to users

- Manage authentication state securely

Visual design is secondary to functional correctness and security.

6. Integration and Validation

The completed system must demonstrate:

- Full frontend–backend integration
- Backend enforcement of business rules
- Correct handling of invalid input and unauthorized access
- Persistent data storage across sessions
- Stable behavior in deployed form

7. Project Phases

Phase I – Backend Foundation

- Business case definition
- Backend project setup
- Core API routes
- Initial data models
- GitHub repository initialization

Phase II – Database and Security Integration

- MongoDB integration using Mongoose
- Authentication and authorization implementation
- Role-based access enforcement
- Secure data handling

Phase III – Frontend Integration and Deployment

- React frontend integration
- Full-stack connectivity
- Deployment to a hosting platform
- Production environment configuration

8. Course Project Presentation

8.1 Purpose of the Course Project Presentation

The course project presentation serves as a formal academic and technical articulation of the completed project. Its purpose is to provide students with the opportunity to **explain, justify, and contextualize** their design decisions, architectural choices, and implementation outcomes in relation to the project requirements and course learning outcomes.

The presentation supports the evaluation of **technical understanding, professional communication, and individual accountability**, and complements the assessment of the submitted codebase and deployed application.

8.2 Presentation Format

Each project group must deliver a **live, in-class course project presentation**. The presentation must be conducted by all group members and must be delivered in a professional academic manner appropriate for an applied computing course.

The use of visual aids or presentation software is permitted but optional. Regardless of format, the presentation must be **structured, concise, and technically focused**.

8.3 Required Presentation Content

The course project presentation must address, at minimum, the following areas:

1. **Project Overview and Business Context**
    - Description of the selected business problem
    - Scope and objectives of the system
    - Identification of user roles

2. **System Architecture**
    - Overall system architecture
    - Separation of frontend, backend, and database components
    - Interaction between system layers
3. **Backend Design**
    - API structure and endpoint responsibilities
    - Use of controllers, services, and middleware
    - Error-handling and validation strategies
4. **Database Design**
    - Overview of collections and relationships
    - Data validation and integrity enforcement
5. **Security Implementation**
    - Authentication workflow
    - Authorization and role-based access control
    - Protection of sensitive resources
6. **Frontend Integration**
    - Interaction with backend APIs
    - Handling of authentication state
    - Role-based application behavior
7. **Deployment**
    - Deployment environment
    - Configuration management
    - Accessibility of the deployed system
8. **Challenges and Technical Decisions**
    - Key challenges encountered
    - Design trade-offs and resolution strategies

8.4 Presentation Expectations

- All group members must participate actively in the presentation.
- Students must be able to explain any part of the system they present.
- Claims made during the presentation must be verifiable through the deployed application or source code.
- The presentation must demonstrate **understanding**, not scripted repetition.

8.5 Academic and Professional Standards

The presentation must reflect:

- Accurate technical terminology
- Clear and logical explanation
- Alignment with the submitted implementation
- Professional academic communication standards

Failure to demonstrate understanding of presented material may result in **individual grading adjustments**, regardless of group performance.

9. Evaluation Criteria

9.1 Backend Architecture and API Design (8%)

- Architectural separation and organization
- RESTful API correctness
- Error handling and configuration management

9.2 Database Design and Data Integrity (6%)

- Schema quality and validation
- Data relationships
- Integrity enforcement and performance considerations

9.3 Security Implementation (6%)

- Secure authentication and authorization
- Backend enforcement of security rules
- Proper handling of sensitive data

9.4 Frontend and Backend Integration (5%)

- Correct API communication
- Authorization enforcement during UI interactions
- Stability and error handling

9.5 Deployment and Configuration (3%)

- Successful deployment
- Environment configuration
- Accessibility of deployed system

9.6 Professional Practice and Individual Contribution (2%)

- Documentation quality
- GitHub commit history
- Individual accountability and understanding

9.7 Evaluation Summary

Component	Weight
Backend Architecture and API Design	8%
Database Design and Data Integrity	6%

Component	Weight
Security Implementation	6%
Frontend–Backend Integration	5%
Deployment and Configuration	3%
Professional Practice and Contribution	2%
Total	**30%**

10. Submission Requirements

Each group must submit:

- One private GitHub repository
- Backend and frontend source code
- A README containing:
    - Project overview
    - Setup instructions
    - Environment configuration
    - Deployment details
- A minimum of **5 meaningful commits per student**

11. Academic Integrity

All work must be authored by the group members. Students must be able to explain all aspects of the implementation. Violations will be handled in accordance with Humber's Academic Integrity Policy.

12. Team Structure

Projects must be completed by **groups of five students**. All members are equally responsible. Individual grading adjustments may be applied based on demonstrated contribution.

13. Instructor Support

Instructor support is available for clarification related to architecture, security implementation, API design, and deployment issues. Students are expected to attempt solutions before seeking assistance.